Programmation web dynamique

TP4&5 - Base de données

Exercise 1 - Configuration

Créez un nouveau projet SpringBoot.
Vous aurez besoin des dépendances suivantes :
- Spring Web,
- Thymeleaf,
- Spring Boot DevTools
- **Spring Data JPA**

Ajout au fichier pom.xml

Pour utiliser le SGBD sqlite, **vous avez besoin d'ajouter manuellement** les dépendances suivantes dans le fichier pom.xml :

```
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
</dependency>
<dependency>
   <groupId>org.hibernate.orm</groupId>
   <artifactId>hibernate-community-dialects</artifactId>
</dependency>
```

Configuration : application.properties

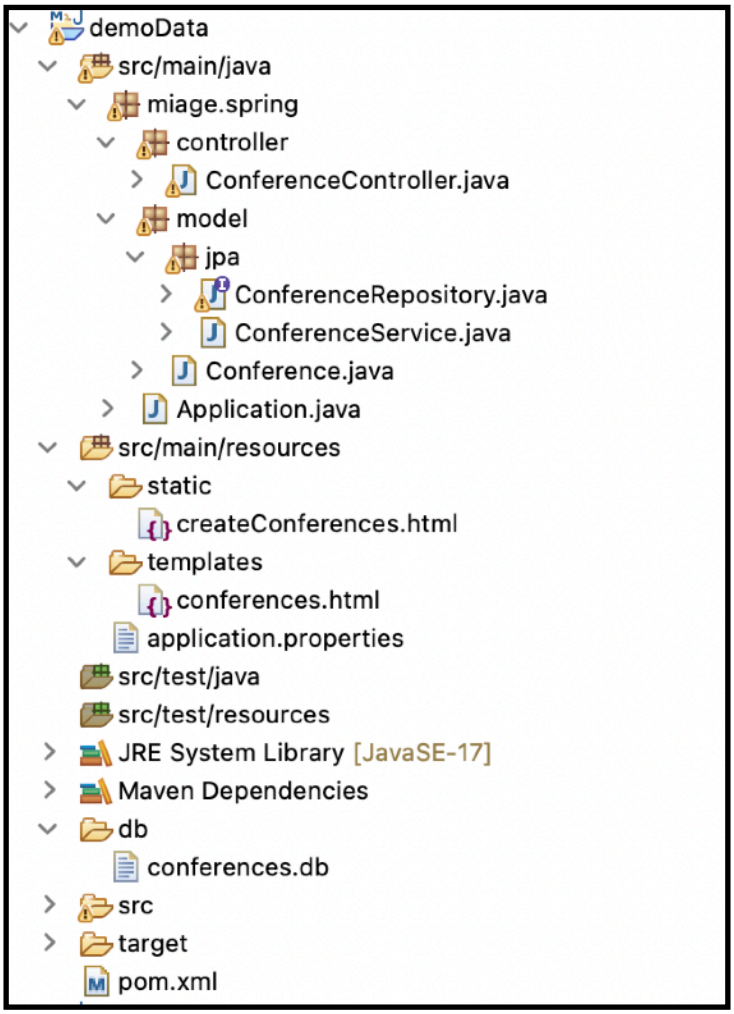
Dans le fichier application.properties, **ajoutez les lignes suivantes pour gérer le SQLite** :

```
# Configuration of the SGBD SQLite
spring.datasource.url=jdbc:sqlite:db/conferences.db
spring.datasource.driver-class-name=org.sqlite.JDBC
spring.datasource.username=
spring.datasource.password=

# Configuration of Hibernate
spring.jpa.database-
platform=org.hibernate.community.dialect.SQLiteDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Organisation des fichiers

Votre projet devrait ressembler à l'image ci-dessous à la fin du TP.
Créez un dossier nommé db (à la racine de votre projet) pour accueillir le fichier de BD SQLite.



Le fichier conferences.db sera créé automatiquement. Pour le visualiser, vous pouvez utiliser l'application web https://sqliteonline.com/.

Exercise 2 - Première classe

Soit le schéma relationnel de la première classe à implémenter (la même que vu en cours):

```
CONFERENCE(idconf, titleconf, nbeditionconf, dtstartconf,
dtendconf, urlwebsiteconf)
```

1. Créez la classe Java Conference correspondant à la table CONFERENCE ci-dessus.
2. Annoter la classe et en particulier l'attribut idconf.
   Utilisez:
   @GeneratedValue(strategy = GenerationType.AUTO) (Spécifique à sqlite)

3. Créez l'interface ConferenceRepository:
   1. Ajouter la méthode permettant de récupérer les conférences par leur titre.
4. Créez la classe ConferenceService, contenant les méthodes:
   1. Renvoie toutes les conférences.
   2. Renvoie les conférences par leur titre.
   3. Ajoute une nouvelle conférence dans la base de données à partir d'une conférence passée en paramètre.

Exercise 3 - Website

Vous allez développez le site web permettant d'implémenter les différentes opérations du CRUD.

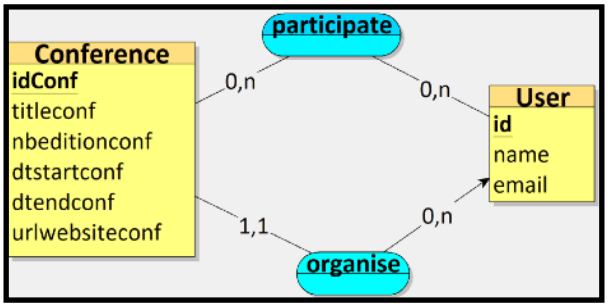
1. Créez une page statique de menu contenant 4 liens hypertextes vers les routes gérant les 4 opérations du CRUD.
2. Pour chacune, vous développerez une page statique demandant des informations à l'utilisateur (formulaires), envoyant les données vers une route spécifique du serveur, réalisant l'opération demandée et renvoyant une page listant toutes les conférences et un message décrivant le résultat de l'opérations.

- Note : Vous pourrez avoir besoin de modifier ConferenceRepository, Conference Service ou les deux.

Exercice 4 - Ajout des Participants

Soit le schéma Entité-Association ci-dessous.

1. Ajouter la classe User et implémenter les associations participer et organiser.
2. Ajouter un repository, un service et un contrôleur permet de gérer les requêtes sur les utilisateurs.
3. Créez :
   (i) Le formulaire HTML et la méthode Java permettant de créer un User et de le stocker dans la base de données.
   (ii) La page HTML et la méthode Java permettant d'afficher tous les utilisateurs avec les conférences qu'ils organisent.
   (iii) La page HTML et les méthodes Java affichant la liste des conférences auquel un utilisateur (identifié par son id) est inscrit pour participer.



Exercice 5 - Schéma de Base de Données complet

Le schéma relationnel précédent est augmenté pour être plus complet.

```
STATUTS(codeStatut, nomStatut)
THEMATIQUES(idThematique, nomThematique)
CONFERENCE(idConf, titreConf, nbeditionconf, dtstartconf,
dtendconf, urlwebsiteconf)
USER(id, name, email, password, codeStatut*)
TRAITER(idConf*, idThematique*)
```

Les conférences traitent maintenant de 1 ou plusieurs thématiques (Informatique, IA, DevOps…). Chaque participant (User) est maintenant caractérisé par un statut (il peut être Etudiant, Chercheur, Industriel). Par ailleurs, à l'inscription d'un utilisateur, un mot de passe est maintenant demandé pour qu'il puisse se connecter au site.

1. Créez les différentes classes permettant d'implémenter ces modifications.
2. Vous pourrez peupler la base de données via des pages HTML que vous créerez ou via une application externe (par exemple, https://sqliteonline.com/)
3. Modifiez vos pages de création et d'affichage des Conferences et User afin qu'elles tiennent compte de la nouvelle structure de Base de Données.
   En prévision de la question suivante, vous assurerez que le champ email soit unique : vous ne pourrez pas créer de nouvel utilisateur si l'email renseigné est déjà présent dans la base de données.
4. **Gestion utilisateur**
   1. Le mot de passe ne doit pas être stocké « en clair » dans une base de données. Il doit être « haché[1] » (transformé) et ce hash est stocké dans la base de données. Pour vérifier si un mot de passe entré par l'utilisateur est correct, il est égalé « haché » et sa transformation est comparée à la valeur stockée en base de données.
      Explorez la librairie BCrypt de Spring Security afin de stocker la version haché du mot de passe en base de données.
   2. Implémentez la possibilité pour l'utilisateur de se connecter au site (avec son email comme identifiant et son password). Si la connection est correcte, l'utilisateur est amené vers une page affichant les conferences qu'a organisé un User et celles auxquelles il participe. L'information de savoir si un utilisateur est connecté et quel utilisateur est connecté sera stocké dans la session.

---

[1] Un **algorithme de hachage** est une fonction mathématique à **sens unique** : elle transforme une donnée d'entrée en une empreinte de taille fixe appelée **hash**.
"monMotDePasse123" ⟶ $2a$10$Xl0yhvzLlaJCzeQgXqMn8O...